

SEMESTER PROJECT REPORT

Title: DevExplorer – Developer-Oriented File Explorer

Author: Asad Ullah (3482-2024)

1. Overview

DevExplorer is a developer-oriented, high-performance file explorer designed and implemented as a Data Structures & Algorithms (DSA) semester project. The primary objective of the project is to bridge the functionality gap between Linux-native file exploration tools (such as *find*, *grep*, and *tree*) and the comparatively limited Windows ecosystem by providing a minimal, fast, and extensible file explorer tailored for developers.

Unlike traditional GUI-heavy file explorers, DevExplorer emphasizes algorithmic efficiency, modular design, and internal indexing. At its core lies a **Hybrid Abstract Data Type (HybridADT)** that encapsulates all major data structures and logic required for directory traversal, file indexing, and searching. The system is initially validated using a CLI-based controller before GUI integration.

2. Problem Statement & Motivation

Standard Windows file exploration tools lack:

- Fast prefix-based filename search
- Content-aware searching without external tools
- Persistent indexing
- Developer-centric utilities (path resolution, ignore rules, code-mode behavior)

Linux systems provide these features natively through shell commands and open-source utilities. DevExplorer aims to replicate and unify these capabilities within a single application while remaining simple enough to implement and evaluate within an academic DSA context.

3. Core Data Structures (HybridADT Design)

All primary logic is encapsulated inside **HybridADT.java**, ensuring a clean separation between data structures and higher-level modules.

3.1 Directory Tree (Iterative Deepening DFS)

The directory hierarchy is indexed using **Iterative Deepening Depth-First Search (IDDFS)**:

- Combines BFS-like completeness (finds shallow files quickly)
- Retains DFS-like memory efficiency
- Prevents excessive memory usage on large file systems

Each directory and file is represented through FileMetadata records and stored in a tree-like logical structure.

3.2 Trie Index (Filename Search)

A **Trie (Prefix Tree)** is used for:

- Case-insensitive filename prefix searching
- Fast lookup ($O(k)$ where k is prefix length)

Each terminal node maintains references to associated file metadata.

3.3 Inverted Index (Content Search)

An **Inverted Index** maps tokens to posting lists containing:

- File references
- Term frequencies

Binary files are safely ignored. This enables efficient full-text search across indexed files.

3.4 Persistence Layer

The complete index state (directory tree, trie, inverted index, metadata, ignore rules) is serialized to disk using **Java Object Serialization**, allowing:

- Fast startup
- No need for reindexing on every launch

4. Abstract Data Type (ADT) Interface

The HybridADTInterface defines **28 operations**, grouped as follows:

- **Navigation Operations:** directory opening, traversal, listing
- **Search Operations:** filename search, content search, filtering, ranking
- **Indexing Operations:** build, update, delete, reindex, persistence
- **Developer Utilities:** code mode, ignore rules, path utilities
- **System Control:** background indexing, event handling, shutdown

This strict interface ensures modularity, testability, and future GUI compatibility.

5. Modular System Architecture

While HybridADT contains all core logic, the application is decomposed into semantic modules that *use* the ADT:

5.1 Navigation Module

- NavController
- PathHistory

Handles directory traversal and navigation state.

5.2 Searching Module

- FileSearch (Trie-based)
- ContentSearch (Inverted index)
- SearchResultRanker

Provides developer-grade search capabilities.

5.3 Indexing Module

- IndexManager
- IndexingPersistence
- IgnoreRules

Manages index lifecycle and filesystem events.

5.4 Developer Tools Module

- PathUtility
- CodeModeController
- TerminalLauncher

Adds utilities commonly required by developers but missing from standard Windows tools.

5.5 System Module

- BackgroundIndexer
- ThreadCoordinator
- ShutdownHandler

Ensures concurrency safety and graceful termination (maximum two threads enforced).

6. Application Control Flow

1. **Startup:** Load persisted index (if available)
2. **Indexing:** Build or update directory index
3. **Navigation/Search:** User interacts via CLI (later GUI)
4. **Event Handling:** File create/modify/delete events update indices
5. **Persistence:** Index state saved on shutdown

The CLI controller DevExplorer.java acts as the orchestrator and validation harness.

7. Verification & Testing

A comprehensive **Full ADT Validation Mode** was implemented in DevExplorer.java, automatically invoking every ADT method:

- Directory indexing and traversal
- Filename and content searching
- Filtering and ranking
- File create/modify/delete updates
- Reindexing
- Persistence save/load
- Background indexing
- Shutdown behavior

All operations were validated using temporary directories to ensure isolation and correctness.

9. Conclusion

DevExplorer successfully demonstrates how advanced file exploration capabilities can be modeled using classical data structures such as trees, tries, and inverted indices. By encapsulating all logic within a Hybrid Abstract Data Type and validating it through a robust CLI, the project achieves both academic rigor and practical relevance. The system is now backend-complete and ready for GUI integration, fulfilling all objectives of the DSA semester project.